

Complete Technical Delivery Report

Detailed implementation, database, security, workflow, and deployment record through 19 September 2026.

Scope	React/Vite frontend, Node.js/Express backend, Supabase, Cloudflare R2, Nodemailer
Status	Implementation complete; database migrations require application in Supabase before deployment
Validation	Frontend production build and backend syntax checks completed

1. Executive Delivery Summary

The requested enhancements were delivered as targeted additions to the existing application. No secrets, environment credentials, or existing core visual identity were replaced.

Domain	Implemented result
Identity	Generated client credentials, bcrypt password hashing, administrator-confirmed resets, masked credential UI
Client project	Structured onboarding data, project overview consistency, protected client contract signing
Commercial flow	Build My Package selections included in Nodemailer enquiry emails
Media	Vendor consent, vendor registry, vendor-linked uploads, grouped shared-media access
Travel	Role-protected travel editing, persisted pricing configuration, fee recalculation and no-invoice fallback

Repository structure

The work spans two repositories that are deployed together.

- latestMidori: React, TypeScript, Vite, Tailwind-style utility classes and API client functions.
- MidoriBackend: Express routing, Supabase data access, JWT middleware, R2 signed URLs and Nodemailer delivery.
- Supabase is the system of record for clients, project steps, invoices, files, vendor assignments and conversation data.

2. Frontend Implementation

The frontend was extended without replacing the existing application navigation or design language.

Client onboarding and project presentation

Client information is now treated as structured project data rather than ad hoc display values.

- Client and event fields are surfaced through project APIs so the overview can render the project record consistently.
- The project detail view incorporates role-aware controls: client actions are visually limited to client-authorized actions while operations controls remain team-facing.
- Existing assets and design components were preserved; the production build resolves all currently referenced image assets.

Credential experience

The UI prevents casual disclosure of generated client passwords.

- Temporary client credentials are initially masked with dot-style presentation.
- A password reveal/reset flow asks for the logged-in administrator's own password before requesting a new temporary password.
- The generated client password is returned only for the creation/reset response and is not subsequently exposed from database reads.

Accessibility and resilience

New dialog behavior was made safer for keyboard and error use cases.

- Consent and upload dialogs include ARIA dialog metadata, keyboard Escape handling and focus restoration.
- Vendor media includes loading, error, empty, video and missing-preview states rather than rendering broken image tiles.
- Rate-limited sequential download handling avoids launching a burst of browser downloads at once.

3. Backend and API Architecture

Express endpoints now enforce the business rules that are represented in the frontend.

Authentication and authorization

Access checks were expanded from simple route visibility to resource-level server enforcement.

- JWT identity is resolved from an HTTP-only cookie or bearer token. Member and client records are loaded with minimal public fields.
- `requireClientAccess` restricts clients to their own `client_id` and restricts team members to projects where they are assigned. Admin and superadmin retain full operational access.
- Dedicated `clientOnly`, `teamOnly`, `adminOnly`, `requireFileAccess` and `requireProjectStepAccess` middleware protect the relevant state-changing endpoints.
- Helmet and request-rate-limit middleware are configured for server hardening, including auth and public enquiry protection.

Core endpoint groups

Implementation includes the following API responsibilities.

- Client creation and credential reset endpoints generate secure temporary passwords and store bcrypt hashes.
- Project routes manage workflow status, moodboard items, production, invoices, client notes, contracts, travel and file downloads.
- Vendor routes list vendors, create vendors, record consent and expose client-scoped vendor media.
- Upload routes accept multipart asset metadata and ensure project-level client access before persistence.

4. Client Credentials and Contract Security

Identity-sensitive actions are protected in both the UI and server layers.

Password storage and reset

The account implementation avoids plaintext persistence for new or reset passwords.

- Client creation uses cryptographically random temporary passwords and hashes them using bcrypt before database insertion.
- The authentication layer supports safe migration of legacy plaintext hashes by verifying existing records and transparently upgrading verified legacy values where applicable.
- Client password reset is restricted to authorized administrators and verifies the administrator password before issuing a replacement temporary password.
- The backend avoids returning password hashes in ordinary client/project responses.

Contract signing

A contract signature is treated as a client authorization, not a general project edit.

- The sign-contract route requires userAuth, requireClientAccess and clientOnly middleware.
- The controller verifies that the signed-in client is signing its own client record.
- Non-client users do not receive a client signing interaction in the project UI.

Security outcome

The result is defense in depth: visual gating guides normal users, while API middleware prevents direct unauthorized requests.

- Client identity cannot be used to reach another client record by substituting a route identifier.
- Team members cannot mutate team-only operational fields without project assignment access.
- Short-lived R2 URLs and authenticated download routes reduce unauthenticated media exposure.

5. Build My Package and Email Delivery

The package builder now passes commercial configuration through to the server-side email workflow.

Frontend payload

At completion, the package form submits a complete enquiry payload to POST /upload/sendenquiry.

- Contact name, email, international phone number and event location.
- Service/project type, selected package name, package feature list and project-scope summary.
- Selected add-ons with calculated pricing, quoted total and budget representation.
- Event timeline/date, vision/notes and additional location details.

Nodemailer implementation

The backend SendEnquiry controller validates data and renders a readable internal enquiry email.

- Nodemailer uses the configured Gmail transporter with EMAIL_SENDER and PASSWORD_SENDER environment variables; these were not changed.
- Recipient is ENQUIRY_RECIPIENT when configured, otherwise the existing Midori company mailbox fallback is used.
- The sender is escaped and used as replyTo, allowing the team to respond directly to the enquirer.
- HTML escaping is applied to all interpolated values to prevent submitted text from being rendered as email markup.

Delivery controls

The enquiry route is rate limited and returns a success/failure API response so the frontend can show appropriate confirmation.

- Validation requires a name and syntactically valid email before a send is attempted.
- Transport verification and transporter error handlers provide operational diagnostics without disclosing secrets to the client.

6. Vendor Media Access - Functional Model

Vendor Media Access is implemented as consent-gated, client-scoped media sharing.

Consent flow

Consent is persisted on the client record rather than stored as an ephemeral frontend preference.

- A client opening Vendor Media Access is shown a consent dialog only while `clients.vendor_media_consent` is false.
- Accepting sets `vendor_media_consent = true` and persists `vendor_media_consent_at` with the acceptance timestamp.
- Declining closes the dialog and returns to Client Assets; acceptance is required before the client media endpoint returns shared vendor media.
- Admins and team members can access the operational vendor view without being presented as a client consent workflow.

Vendor record workflow

Vendors are client-scoped entities, created from the upload workflow only by administrators.

- The upload UI requires a vendor selection whenever `Is this for vendor` is enabled.
- Existing vendors are shown in a selector. Administrators can create a new vendor inline; non-admins receive an explanatory authorization message.
- Vendor creation captures `vendor_name`, `vendor_type` and optional contact details and records who created the vendor.

7. Vendor Media - Storage and Retrieval

The media relationship is normalized in Supabase and enforced server-side.

File association

A vendor-shared file always belongs to exactly one vendor for the current client.

- The upload controller accepts `vendor_id` together with `is_vendor_shared`.
- For vendor-shared uploads, the backend checks that the selected vendor exists and belongs to the upload's `client_id`.
- Non-vendor files carry `vendor_id` as NULL.
- The SQL trigger independently rejects a shared file without `vendor_id` and rejects a vendor assigned to a different client.

Media retrieval and presentation

Vendor Media Access fetches the minimum necessary information.

- The backend selects files where `client_id` matches, `is_vendor_shared` is true and `vendor_id` is non-null.
- Files are grouped by vendor and zero-file vendor groups are removed before the response.
- Each valid object storage key receives a short-lived signed R2 preview URL.
- The frontend displays vendor name, type, file count, open and download actions. Legacy expiry/permission display was removed from the live view.

Downloads

Single-file downloads use existing authenticated download endpoints. Download all sequences files with a controlled delay to avoid simultaneous browser-triggered downloads.

- Signed URLs have bounded lifetimes and are generated server-side.
- Errors remain visible to the user through the existing toast/error-state patterns.

8. Crew Travel - Completed Remediation

The Crew Travel review identified and corrected authorization, configuration persistence and no-invoice behavior.

Travel editing controls

Travel changes are now operations-only server actions.

- GET travel data remains visible to authorized project users, including clients where appropriate.
- PUT /project/travel/:clientId/:projectStepId now requires teamOnly in addition to project/client authorization.
- The frontend hides travel row editing and pricing-setting controls for client users.
- Venue must be a non-empty string no longer than 250 characters. Distance must be finite, non-negative and within a reasonable upper bound.

Persisted pricing configuration

The free allowance and mileage rate are client-level database settings, not temporary UI values.

- PUT /project/travel/:clientId/config stores travel_free_allowance and travel_rate_per_mile.
- Configuration values are validated as finite, non-negative numeric values.
- Travel step retrieval includes travel_config, allowing the UI to apply server-backed values with defaults of 20 miles and \$0.70 per mile when legacy records lack values.

Fee and invoice fallback

Changing a location or pricing setting recalculates total miles, billable miles and travel fee.

- Client driving_distance and travel_fee are synchronized.
- When an invoice exists, travel_fee, final_amount and amount_due are updated from the recalculated amount.
- When no invoice exists, the travel step and client totals are saved successfully and the API returns invoice_updated: false rather than failing.
- The UI has explicit loading and no-location fallback states.

9. Supabase Schema and Tables

The following tables and columns are involved in the delivered functionality. Apply the consolidated migration before deployment.

Table	Relevant columns and use
clients	client_id, client_name, email, event fields, password hash, driving_distance, travel_fee, travel_free_allowance, travel_rate_per_mile, vendor_media_consent, vendor_media_consent_at
members	member_id, full_name, email, role, password_hash; used for member authentication and vendor creator attribution
project_steps	project_step_id, client_id, step_name, step_order, venue, travel_distance, assigned_member_ids, scheduled_time
files	file_id, client_id, file_name, file_type, object_storage_key, is_vendor_shared, vendor_id
vendors	vendor_id, client_id, vendor_name, vendor_type, contact_email, contact_phone, created_by, created_at, updated_at
invoices	invoice_id, client_id, subtotal_amount, tax_amount, discount_amount, travel_fee, final_amount, amount_due, amount_paid
travel_discussions	travel_discussion_id, client_id, member_id, message, message_by_role, created_at
moodboard_songs	Existing data plus artist and notes fields added by the migration

Important indexes

Indexes are included for common client/project and vendor/file query paths.

- project_steps(client_id), files(client_id), invoices(client_id), gears_assigned(client_id, member_id).
- vendors(client_id) and files(vendor_id) for vendor-media grouping and filtering.

10. Supabase Migration Specification

The migration is located at MidoriBackend/supabase/migrations/20260916_project_data_hardening.sql.

Client columns

The migration performs idempotent ADD COLUMN IF NOT EXISTS operations.

- event_date DATE, event_end_date DATE and event_location TEXT.
- vendor_media_consent BOOLEAN NOT NULL DEFAULT false and vendor_media_consent_at TIMESTAMPTZ.
- travel_free_allowance NUMERIC NOT NULL DEFAULT 20 and travel_rate_per_mile NUMERIC NOT NULL DEFAULT 0.70.
- Existing driving_distance and travel_fee should be present; the latest check query safely adds them if missing.

Vendor table DDL

vendors is a normalized, client-owned table.

- vendor_id UUID primary key with gen_random_uuid default.
- client_id foreign key to clients with ON DELETE CASCADE.
- created_by foreign key to members; optional vendor contact fields and audit timestamps.
- files.vendor_id is a foreign key to vendors with ON DELETE SET NULL, retaining file history if a vendor is removed.

Database trigger

validate_vendor_shared_file is created on files before insert/update of is_vendor_shared or vendor_id.

- If is_vendor_shared is true, vendor_id is required.
- The trigger queries vendors to ensure the vendor client_id equals the file client_id.
- If a file is not vendor shared, the trigger clears vendor_id to keep the relationship consistent.

11. Required Supabase Checks and Deployment Order

The release is code-complete, but the database migration is a prerequisite for the new vendor and travel functionality.

Columns to verify or create

Run the latest migration. At minimum confirm these columns exist.

- clients: vendor_media_consent, vendor_media_consent_at, travel_free_allowance, travel_rate_per_mile, driving_distance, travel_fee.
- project_steps: venue and travel_distance.
- files: is_vendor_shared and vendor_id.
- vendors table and its vendor_id/client_id relationship.

Historical data consideration

Old vendor-shared files might not have a vendor association because that historical relationship cannot be inferred safely.

- Inspect files where is_vendor_shared = true and vendor_id IS NULL after migration.
- Create/choose the correct vendor and assign vendor_id during a controlled cleanup. New writes are protected by the backend and trigger.
- Do not use a broad destructive update; review each historic assignment against project context.

Recommended release order

Deploy frontend and backend as a matched release after the schema update.

- Back up the database schema and apply migration in Supabase SQL Editor or migration runner.
- Deploy MidoriBackend, then latestMidori build artifacts.
- Run role-based smoke tests for admin, assigned team member and client accounts.
- Verify Nodemailer environment configuration in the target deployment without exposing or rotating existing secrets.

12. Validation, Limitations and Handover

The following checks were performed against the local codebase.

Completed validation

Static and build verification was completed after the final Crew Travel fix.

- Vite production build completed successfully with 2,391 modules transformed.
- Node syntax checks passed for ProjectDetails.controller.js, Vendor.controller.js, route modules and Express startup modules.
- Git diff whitespace validation passed for the modified repositories.
- The original image assets resolve in the current production build.

Operational limitations

These are deployment checks, not code blockers.

- Live browser automation was unavailable because the local agent-browser executable was not installed; production role-based click-through should be run after deployment.
- Email delivery requires the existing sender credentials and recipient configuration to be valid in the deployment environment.
- Supabase row-level security policies, if enabled, must permit the service role/API access patterns used by the backend.

Final handover state

The application now has server-enforced access rules for sensitive client workflows, structured vendor media sharing, complete package enquiry email content, and resilient crew-travel calculations with persistent configuration.

- No secrets were changed or rotated.
- No assets were deliberately removed.
- The remaining deployment action is applying the Supabase migration and completing the recommended smoke test.